

Docente del curso: Benner Humberto Ojeda Ríos

Nombres y Apellidos del estudiante: Jesús Valentín Niño Castañeda

Fecha: 29/01/2025

Sección: 1

Nota: 11.75

Indicaciones específicas:

- Duración: 80 minutos
- La evaluación consta de 3 preguntas.
- En caso de haber soluciones similares de preguntas serán calificados con cero (0).
- Sea ordenado y legible en la resolución de cada ejercicio, caso contrario se invalidará su respuesta.

Teoría: 8.5
Práctica: 2.0
P.E: 1.25

Resuelva:

1. (3.5 pts points) **General: Verdadero o falso**

Para cada afirmación a continuación, indique si es **verdadera o falsa**. Explicar.

1. (0.5 pts) La búsqueda binaria en un arreglo ordenado es el peor de los casos $O(n)$.

0.0 Falso, el binary search en un array ordenado es siempre $\theta(\log_2 n)$ $O(\log n) \leq O(n)$

2. (0.5 pts) El número de colisiones en una tabla hash depende únicamente de la capacidad de la tabla y de la función hash.

0.0 X

3. (0.5 pts) Una tabla hash garantiza un tiempo de búsqueda constante.

0.5 Falso, solo es constante cuando no hay colisiones lo cual no se garantiza por lo que en el peor de los casos es $\Omega(n)$?

4. (0.5 pts) Si cada nodo en un árbol binario tiene 0 o 2 hijos, entonces la altura del árbol es $\Theta(\lg n)$.

0.5 X

5. (0.5 pt) Un sistema operativo necesita programar cuándo ejecutar cada uno de los procesos actuales. Una pila es la estructura de datos más apropiada para resolver el problema anterior.

0.5 Falso, sería más apropiado utilizar una estructura FIFO como una cola para que los procesos se ejecuten en el orden solicitado.

6. (0.5 pt) Quiere almacenar 1 MB de datos no comparables y espera ejecutar la operación de búsqueda con mucha frecuencia. Un árbol búsqueda binaria balanceado es la estructura de datos más apropiada para resolver el problema anterior.

0.5 Falso, el algoritmo de inserción de un BST requiere poder comparar los datos para poder distribuirlos ordenadamente en su estructura.

Usar Hash.

7. (0.5 pt) Si el factor de carga de una tabla hash es menor que 1, entonces no hay colisiones.

0.5 Falso, no se puede asegurar ya que las colisiones también dependen de otros factores como la función hash.

$$c) f_4(n) \in \Theta(n^2), n^2 \in O(2^n) \Rightarrow f_4(n) \in O(2^n)$$

$$f_1 \in \Theta(n^{1.5}), n^{1.5} \in O(n^2) \Rightarrow f_1 \in O(n^2) \in O(f_2(n))$$

$$\in O(f_4(n))$$

$$f_1 \leq f_4 \leq f_3 \leq f_2$$

2. (5.5 points) Notación asintótica

(a) (1 pt) Considere la función $f(n) = (\sin(n) + 2)n$, donde $\sin(n)$ está acotada entre -1 y 1 para todo n . Pruebe que $(\sin(n) + 2)n \in \Theta(n)$ usando las propiedades de límites.

(b) (2 pt) Pruebe la siguiente sentencia usando la definición formal de Θ :

$$\sum_{i=1}^n \sum_{j=1}^i j = \Theta(n^3)$$

(c) (2.5 pt) Ordene las funciones en orden creciente de complejidad asintótica (notación O-grande). Justifique su respuesta.

$$f_1(n) = n\sqrt{n}, f_2(n) = 2^n, f_3(n) = n^{10} \cdot 2^{n/2}, f_4(n) = \sum_{i=1}^n (i+1)$$

Solución:

$$a) (\sin(n) + 2)n \in \Theta(n)$$

$$\lim_{n \rightarrow \infty} \frac{n(\sin(n) + 2)}{n} = \lim_{n \rightarrow \infty} \sin(n) + 2$$

$$-1 \leq \sin(n) \leq 1, \forall n$$

$$1 \leq \sin(n) + 2 \leq 3$$

$$\Rightarrow 1 \leq \lim_{n \rightarrow \infty} \sin(n) + 2 \leq 3$$

$$0 < \lim_{n \rightarrow \infty} \sin(n) + 2 < \infty$$

$$b) \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \sum_{i=1}^n i^2 + i = \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right) = \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) =$$

$$\frac{1}{4} \left(\frac{(n^2+n)(2n+1)}{3} + n^2+n \right) = \frac{1}{12} (2n^3 + n^2 + 2n^2 + n + 3n^2 + 3n) = \frac{2n^3 + 6n^2 + 4n}{12}$$

$$\lim_{n \rightarrow \infty} \frac{(2n^3 + 6n^2 + 4n)^{1/3}}{(12n^3)^{1/3}} = \lim_{n \rightarrow \infty} \frac{2}{12} = \frac{1}{6}$$

$$c) f_1(n) = n^{3/2}, f_4(n) = \sum_{i=1}^n i + \sum_{i=1}^n 1 = \frac{n(n+1)}{2} + n = \frac{n(n+3)}{2} = \frac{n^2 + 3n}{2}$$

$$\lim_{n \rightarrow \infty} \frac{(2^n)^{1/2}}{(n^{3/2})^{1/2}} = \lim_{n \rightarrow \infty} \frac{(2^n \cdot \ln(2))^{1/2}}{(\frac{3}{2} n^{3/2})^{1/2}} = \lim_{n \rightarrow \infty} \frac{\ln^2(2) \cdot 2^n}{\frac{3}{4} n^{3/2}} = \frac{4}{3} \ln^2(2) \lim_{n \rightarrow \infty} \sqrt{n} \cdot 2^n = \infty$$

$$f_2(n) \in \Omega(f_1(n)) \Rightarrow f_1(n) \in O(f_2(n))$$

$$\lim_{n \rightarrow \infty} \frac{n^{10} \cdot 2^{n/2}}{2^n} = \lim_{n \rightarrow \infty} n^{10} \cdot 2^{-n/2} = \lim_{n \rightarrow \infty} \frac{n^{10}}{2^{n/2}} = \lim_{n \rightarrow \infty} \frac{10!}{\ln^{10}(2) \cdot 2^{n/2}} \cdot \frac{1}{2^{10}} = \frac{10! \cdot 2^{10}}{\ln^{10}(2)} \lim_{n \rightarrow \infty} \frac{1}{2^{n/2}} = 0$$

$$f_3(n) \in O(f_2(n))$$

LFUCache(int capacity)

head = new Node

Node* temp = head

for (i=0; i<capacity; ++i) {

temp->next = new Node

temp->next->prev = temp

3. (5 points) Hashing

```
temp = temp->next  
}  
tail = new Node;  
temp->next = tail  
tail->prev = temp
```

a) Diseñe una estructura de datos para un caché *Least Frequently Used* (LFU), es decir, Menos Frecuentemente Usado. Diseñe los algoritmos para cada función de la clase LFUCache:

- LFUCache(int capacity): Inicializa el objeto con la capacidad de la estructura de datos.
- (2.5 pts) int get(int key): Obtiene el valor de la clave key si la clave existe en el caché. De lo contrario, devuelve -1.
- (2.5 pts) void put(int key, int value): Actualiza el valor de la clave key si está presente, o inserta la clave si no está presente. Cuando el caché alcanza su capacidad, se debe invalidar y eliminar la clave menos frecuentemente usada antes de insertar un nuevo elemento. En este problema, si hay un empate (es decir, dos o más claves con la misma frecuencia), se invalidará la clave menos recientemente usada.

Para determinar la clave menos frecuentemente usada, se mantiene un contador de uso para cada clave en el caché. La clave con el contador de uso más bajo es la menos frecuentemente usada. Cuando una clave se inserta por primera vez en el caché, su contador de uso se establece en 1 (debido a la operación put). El contador de uso de una clave en el caché se incrementa cuando se llama a las operaciones get o put sobre ella. Las funciones get y put deben ejecutarse cada una en **tiempo promedio $O(1)$** .

Solución:

```
int get(int key)  
{  
    if (map.find(key) != map.end())  
        return map[key]  
    return -1  
}
```

Incomplete!

Docente del curso: Benner Humberto Ojeda Rios

Nombres y Apellidos del estudiante: Jesús Valentín Niño Castañeda

Fecha: 29/01/2025 Sección: 1 Nota: _____

Indicaciones específicas:

- Duración: 30 minutos
- En caso de haber soluciones similares de preguntas serán calificados con cero (0).

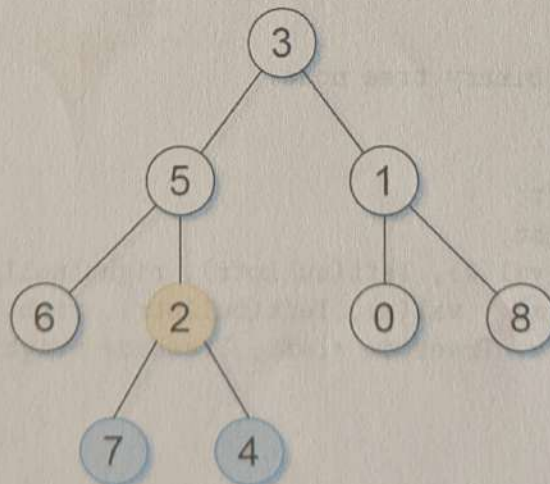
Resuelva:

1. (6 pts) Dada la raíz de un árbol binario, devuelve el ancestro común más bajo de sus hojas más profundas.

Recordar que:

- El nodo de un árbol binario es una hoja si y sólo si no tiene hijos.
- La profundidad de la raíz del árbol es 0. Si la profundidad de un nodo es d , la profundidad de cada uno de sus hijos es $d + 1$.
- El ancestro común más bajo de un conjunto S de nodos es el nodo A con la mayor profundidad, de modo que cada nodo en S está en el subárbol con raíz A .

Ejemplo 1:



■ **Entrada:** root = [3,5,1,6,2,0,8,null,null,7,4]

■ **Salida:** [2,7,4]

■ **Explicación:** Devolvemos el nodo con valor 2, coloreado en amarillo en el diagrama.

Los nodos coloreados en azul son los nodos de las hojas más profundos del árbol. Tenga en cuenta que los nodos 6, 0 y 8 también son nodos hoja, pero su profundidad es 2, pero la profundidad de los nodos 7 y 4 es 3.

Ejemplo 2:

■ **Entrada:** root = [1]

■ **Salida:** [1]

■ **Explicación:** La raíz es el nodo más profundo en el árbol, y es el ancestro común más bajo (LCA) de sí mismo.

Ejemplo 3:

■ **Entrada:** root = [0,1,3,null,2]

■ **Salida:** [2]

■ **Explicación:** La hoja más profunda en el árbol es 2. El ancestro común más bajo (LCA) de un único nodo es el mismo nodo.

Restricciones:

■ El número de nodos en el árbol estará en el rango [1, 1000].

■ $0 \leq \text{Nodo.val} \leq 1000$

■ Los valores de los nodos del árbol son únicos.

/** TEMPLATE

* Definition for a binary tree node.

* struct TreeNode {

* int val;

* TreeNode *left;

* TreeNode *right;

* TreeNode() : val(0), left(nullptr), right(nullptr) {}

* TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}

* TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),

* left(left),

* right(right) {}

* };

*/

```

class Solution {
public:
    TreeNode* lcaDeepestLeaves(TreeNode* root) {
        int max;
        vector<Node*> nodes;
    };
}

```

1. Se crean variables max y nodes
 max: maxima profundidad
 nodes: nodos de maxima profundidad
2. Se hace un 1er recortido para hayar la profundidad maxima
3. Se hace un 2do recorrido para hayar los nodos más profundos

Completar algoritmo!